

COMPREHENSIVE PRACTICAL TEST AND SCALABILITY BENCHMARKS OF SORTING ALGORITHMS IN PYTHON

Nichita Gasnas
Department of Computer Science,
West University of Timișoara
email: nichita.gasnas00@e-uvt.ro

Abstract

This paper puts sorting algorithms to the test, not just on paper but with real code running in a high-level language. Sure, textbooks give us the Big-O math— $\mathcal{O}(n^2)$ for the simple sorts and $\mathcal{O}(n \log n)$ for the better ones—but those are just theory. How fast algorithms run in practice has a lot to do with how computers actually work: hardware, memory handling, and the rules the programming language plays by. By testing lists up to 1,000,000 items, mixing things up with different sorts of data (Random, Sorted, Reverse, Mixed, Flat), this study pinpoints where basic algorithms break down and when advanced designs step in. It looks at Parallel Merge Sort, Stream Sort, Quick Sort, classic Merge Sort, and some basic sorts. Beyond that, it dives into how pure Python slows everything—the gap between Python’s code and the raw speed in lower-level languages is a critical finding. The results show some clear limits: recursive depth, the price of pointer-based data structures, when multi-core really starts to matter, and just how crucial memory management is at big scale.

Contents

1	Introduction	3
2	Theoretical Analysis of Sorting Algorithms	3
2.1	Basic Quadratic Sorts	4
2.2	Splitting (Divide-and-Conquer) Architectures	4
2.3	High-Performance Methods	4
3	Implementation Architecture and System Model	4
3.1	System Specifications and Architecture	4
3.2	The Impact of the Language: Interpreter vs. Compiled Code	5
3.3	Memory Problems: Linked Lists in Python	5
3.4	Execution and Usage (User Guide)	5
4	Case Studies/Experiment	5
4.1	Test Data Shapes and Variations	5
4.2	System Crashes at Extreme Scale	6
4.3	The Reality of Time Complexity at Scale	6
4.4	The Parallel Processing Paradox	6
4.5	Handling Duplicates: The Flat Data Check	6
4.6	Complete Algorithm Performance Reviews	7
4.7	Choosing the Best Algorithm by Scenario	8
5	Related Work	9
6	Conclusions and Future Work	9

1 Introduction

This research sets out to highlight the real gap between what the math says about sorting speed and what actually happens when you run code on a machine. In computer science classes, sorting algorithms get measured and compared in “Big-O” notation [1, 2, 3], and that gives a solid rule for how slow things get as inputs grow. But in reality, real speed hinges on the programming language, the structure of the computer’s memory, and how the data is organized. Algorithms don’t run in a math vacuum—they run under strict limits set by the computer and the language.

To dive into this, I built a series of standard sorting algorithms from scratch in pure Python, pushing them right up to their real-world limits. My direct contribution lies in building, running, and comparing these algorithms to clearly map out Python’s physical execution boundaries. The tests include Quick Sort, Merge Sort using arrays, Merge Sort built on linked lists, Insertion Sort, Selection Sort, Bubble Sort, Parallel Merge Sort, and Stream Sort via heaps.

Just to show how all this plays out, let’s keep a simple example running in the background: $A = [8, 3, 5, 1, 9, 2]$. With the basic approaches, your code just churns side-by-side comparisons. More advanced algorithms rethink how they handle the data altogether. By slowly ramping up from tiny lists like this example to one million items, and mixing up the data, this study finds out exactly when simple sorts choke and when more powerful, multi-core methods take over. The paper also digs into how Python’s interpreter slows things down by translating each instruction.

Here’s how things are set up: Section 2 breaks down the tested algorithms and what the math says they should do. Section 3 explains the system model and how Python’s design changes implementation. Section 4 covers what happened in the experiments. Then Section 5 puts the results up against other work, and Section 6 closes with conclusions and next steps.

2 Theoretical Analysis of Sorting Algorithms

To really get at the tests, it’s key to define the sorting problem and the basics of each algorithm. The task: take a sequence and give back a permutation with the data in non-decreasing order. The algorithms range from plain, old-school methods to more advanced divide-and-conquer designs, each bringing their own guarantees for correctness, completeness, and efficiency.

2.1 Basic Quadratic Sorts

Quadratic sorts compare each item with the others; when you double the items, the run time quadruples ($\mathcal{O}(n^2)$). Bubble Sort is the classic tortoise [1, 4]; it moves the biggest numbers to the end, one swap at a time. Selection Sort doesn't care how sorted the data is—it always scans the whole remainder to find the next smallest, every time. Insertion Sort slides items into place instead of swapping everything; if data is nearly in order, it hardly does any work and can get close to $\mathcal{O}(n)$ speed. Back to our example $A = [8, 3, 5, 1, 9, 2]$: Insertion Sort would just slide the 1 past bigger numbers, not swap it step-by-step.

2.2 Splitting (Divide-and-Conquer) Architectures

These sorts break the list up and are much faster in practice; they promise average $\mathcal{O}(n \log n)$ speed [1, 5, 6]. Quick Sort picks a “pivot” (say, 8 from our running example) and splits the list, making big jumps. Merge Sort splits lists right down the middle and merges everything back, performing consistently no matter how jumbled the data starts out.

2.3 High-Performance Methods

Parallel Merge Sort divvies the work up, giving separate pieces to each CPU core [7, 8]. Stream Sort, using a Heap Merge approach, is similar, but it focuses on squeezing down memory use, using a fast priority queue to glue pieces back together in steps so it doesn't need deep repeating functions.

3 Implementation Architecture and System Model

You can't separate algorithm speed from the machine. New CPUs are a lot faster than old ones, so hardware setup and how data is stored matters a lot for real results.

3.1 System Specifications and Architecture

All the benchmarks ran on Windows 11 with an AMD Ryzen 9 6900HS and 32GB of RAM. This especially mattered for the Parallel Merge Sort—how many cores and how much memory you've got directly affects whether you can split the work efficiently and avoid crashing when data gets big.

3.2 The Impact of the Language: Interpreter vs. Compiled Code

It's not just the math. The language changes everything. Python code is easy to read, but it's slow because it's interpreted and does a lot of type-checking. When Python runs, it reads instructions line-by-line [9, 10], which adds overhead to every check and every comparison. Compiled languages like C or C++ translate the logic straight to instructions before anything runs, letting code dig directly into memory.

3.3 Memory Problems: Linked Lists in Python

How you structure data shapes performance. Comparing Merge Sort on arrays vs. Linked Lists showed Python's pain with certain setups. Textbooks make Linked Lists sound great, but in Python, each node is a separate object. Moving through thousands of nodes is a nightmare—pointer chasing and recursion both hit hard. Linked lists spill over Python's recursion limits quickly, while arrays let CPUs skip to locations directly using simple index numbers.

3.4 Execution and Usage (User Guide)

The benchmarking tool was built for easy repeats. You need Python 3.12 or above to run it. Start the script with command line options to set the list size and pick the data shape (e.g., executing `python benchmark.py --size 1000000 --shape random`). It creates the arrays, runs the sorts, then prints out a formatted table detailing the algorithm name, list shape, size, and exactly how long it took, all right on the console.

The complete source code for these benchmarks, along with the raw experimental result logs, is publicly available on GitHub at: <https://github.com/nichitagnas00-dev/Benchmark-source-code-and-results>.

4 Case Studies/Experiment

4.1 Test Data Shapes and Variations

Test lists came in all flavors—small (20, 30, 50, and 100 items), medium (1,000 and 10,000), and large (up to 1,000,000). Data types used included Integers, Floats, and Strings. Since tiny lists sort so fast you can barely measure them perfectly, each small sort was put into a loop that ran 100,000 times to capture what's

really happening. List shapes included: Random, Sorted, Reverse, Almost Sorted, Mixed, and Flat (lists with lots of duplicates to check duplicate-handling).

4.2 System Crashes at Extreme Scale

At first, I wanted to test with tens or hundreds of millions of items. Didn't happen—three things stopped it. First, Python lists eat much more RAM than arrays since they're lists of pointers [10, 11, 12]. At 100 million items, they blew up past the 32GB RAM limit. Second, Python has a hard recursion limit—about 1,000 steps [13]—and heavy recursion on big lists quickly brings a `RecursionError`. Third, Python's own overhead is huge. So, the safe upper limit for reliable data was forcefully capped at one million items.

4.3 The Reality of Time Complexity at Scale

The tests proved what everyone suspects: quadratic $\mathcal{O}(n^2)$ sorts die fast as lists get larger. Bubble Sort, run 100,000 times on just 100 items, took 41 seconds to complete. Once lists hit 10,000 items, I had to put a strict safety rule in to skip the slowest sorts to avoid freezing things. But Insertion Sort stood out. On a “Sorted” 100-item list, it finished 100,000 loops in only 1.55 seconds, clearly beating Quick Sort at 6.54 seconds.

4.4 The Parallel Processing Paradox

Here's a twist: Parallel Merge Sort isn't always better. Running across multiple cores costs something—communication setup, known as Inter-Process Communication (IPC) overhead [7, 14]. For a 10,000-item list, normal single-core Merge Sort clocked 0.0224 seconds, leaving Parallel Merge Sort far behind at 0.4711 seconds. Still, once you hit a million items, the tables turn completely: Standard Merge Sort takes 3.7171 seconds, while Parallel Merge Sort blows through the entire job in just 0.9687 seconds.

4.5 Handling Duplicates: The Flat Data Check

Flat data—almost all duplicates—let Quick Sort really shine. Sorting a one-million-item “Flat” list took Quick Sort just 0.1694 seconds, while purely random data at the exact same size took 3.2622 seconds.

Table 1: Practical Test Results Across Sizes and Shapes

This table tells the real story. It contrasts the slow climb of quadratic growth with the speed of logarithmic efficiency, highlights when Parallel Merge Sort takes over at 10,000 items, and shows how Quick Sort rockets through Flat data.

Algorithm	Items (Size)	List Shape	Time (s)
Bubble Sort	100 ($\times 100k$ loops)	Random	41.0000
Insertion Sort	100 ($\times 100k$ loops)	Sorted	1.5500
Quick Sort	100 ($\times 100k$ loops)	Sorted	6.5400
Merge Sort (1 Core)	10,000	Random	0.0224
Parallel Merge Sort	10,000	Random	0.4711
Merge Sort (1 Core)	1,000,000	Random	3.7171
Parallel Merge Sort	1,000,000	Random	0.9687
Quick Sort	1,000,000	Flat (Many Duplicates)	0.1694
Quick Sort	1,000,000	Random	3.2622

4.6 Complete Algorithm Performance Reviews

Let's step through when each algorithm does best or worst and what makes the difference.

Bubble Sort

Best Case: $\mathcal{O}(n)$ if the list's already sorted and it breaks early.

Worst Case: $\mathcal{O}(n^2)$ on reverse-sorted data.

Review: It's hopeless for big data—too many side-by-side swaps. In tests, it took over 41 seconds for just 100 items (looped 100k times) and was cut off at $N = 10,000$ to avoid freezing the computer.

Selection Sort

Best/Worst Case: $\mathcal{O}(n^2)$ no matter what.

Review: It always checks the rest even if the list's sorted, so it's consistently slow on all data shapes.

Insertion Sort

Best Case: $\mathcal{O}(n)$ on nearly-sorted or tiny lists.

Worst Case: $\mathcal{O}(n^2)$ when reversed.

Review: It slides elements, skips already-sorted sections, and at $N = 100$ (Sorted), it beat Quick Sort handily. Great as a helper step for small pieces.

Quick Sort

Best Case: $\mathcal{O}(n \log n)$ on random or flat data.

Worst Case: $\mathcal{O}(n^2)$ on already-sorted or reversed lists if you pick a bad pivot.

Review: Super fast on Flat data—just 0.1694 seconds for a million items. Recursive function overhead can trip it up in Python for huge lists, often causing a `RecursionError`.

Merge Sort (Array)

Best/Worst Case: Solid $\mathcal{O}(n \log n)$ no matter the data.

Review: Always reliable. Downside: needs $\mathcal{O}(n)$ extra memory. Slicing lists in Python slows it compared to Quick Sort on random data, but it's more stable and less likely to hit recursion limits.

Merge Sort (Linked List)

Review: Sounds good in theory, terrible in practical Python. $\mathcal{O}(n \log n)$ remains, but pointer chasing and node management bring massive memory problems. Always crashed before a million items due to recursion limits, proving solid arrays are a must in Python.

Parallel Merge Sort

Review: The go-to multi-core workhorse for massive data. Coordination makes it slower than normal Merge Sort on smaller data ($N \leq 10,000$), but at a million items it's unbeatable—finishing in under a second.

Stream Sort (Heap Merge)

Review: The survivor algorithm. Uses a tuned priority queue, works step-by-step without repeating functions, and avoids memory blowups. It's the only pure-Python option that handles giant datasets on machines with tight memory.

4.7 Choosing the Best Algorithm by Scenario

From hands-on tests, you pick the algorithm for your context:

- **Tiny datasets** ($N < 50$): *Insertion Sort*. The setup for Quick/Merge Sort outweighs their benefits here—Insertion's sliding is fastest.

- **Nearly sorted data:** *Insertion Sort*. It adapts, often running at straight-line $\mathcal{O}(n)$, outpacing mathematically fancier algorithms.
- **Highly duplicated (Flat) data:** *Quick Sort*. It splits up identicals fast, shrinks the work, and almost hits linear speed.
- **Massive datasets** ($N > 10^5$): *Parallel Merge Sort*. When data's big enough to pay off the coordination between processors, multi-core is vital.
- **Memory-limited computers:** *Stream (Heap) Sort*. If RAM is too tight for list copying (like Merge Sort needs) or recursion (like Quick Sort needs), a heap-based step-wise process sorts fast and safely.

5 Related Work

This work stands alongside classic sorting theories found in foundational textbooks by Cormen et al. [1], Knuth [2, 6], and Aho, Hopcroft, and Ullman [3]. Where those works focus on pure Big-O math, this project zooms in on what really limits algorithms in a modern high-level language. Richard Cole [8] mathematically outlined the theory of Parallel Merge Sort, but this paper physically tests it against actual operating system costs [14, 7]. The memory and scaling issues found here agree with newer studies—for example, the 2026 research by Wuttke et al. [9]—confirming that Python's interpreter is a major speed hurdle.

6 Conclusions and Future Work

This test didn't just stay in the safe zone of textbook math—it clashed head-on with the gritty details of real computing. Python's built-in recursion limits and memory constraints forced a hard cap at a million items, just to keep the computer from crashing. But even within that ceiling, the results speak for themselves: while basic $\mathcal{O}(n^2)$ algorithms work alright for tiny or nearly sorted lists, they just can't compete. Only $\mathcal{O}(n \log n)$ sorts make sense as data sizes grow.

Still, even the best algorithm can hit a wall. When you're sorting millions of records, Python itself becomes the bottleneck—the interpreter just can't keep up. For serious speed at scale, you need smarter approaches, like parallel processing with multi-core algorithms (think Parallel Merge Sort), or you have to ditch Python and code directly in a compiled language. Relying on pure Python for big data jobs just doesn't cut it.

Looking ahead, the obvious next move is to rewrite these exact sorting algorithms in a compiled language like C or C++. By stripping away the Python

interpreter's drag, the test could finally blow past the million-item barrier and see just how big the lists can get before the raw computer RAM gives up.

References

- [1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, third edition, 2009.
- [2] Donald E. Knuth. Big omicron and big omega and big theta. *SIGACT News*, 8(2):18–24, 1976.
- [3] Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman. *The Design and Analysis of Computer Algorithms*. Addison-Wesley, 1974.
- [4] Owen Astrachan. Bubble sort: An archaeological algorithmic analysis. *SIGCSE Bulletin*, 35(1):1–5, 2003.
- [5] Jon Louis Bentley. Multidimensional divide-and-conquer. *Communications of the ACM*, 23(4):214–229, 1980.
- [6] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, second edition, 1998.
- [7] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. Wiley, tenth edition, 2018.
- [8] Richard Cole. Parallel merge sort. *SIAM Journal on Computing*, 17(4):770–785, 1988.
- [9] J. Wuttke et al. Unlocking python’s cores: Hardware usage and energy implications of removing the gil. arXiv preprint arXiv:2603.04782, 2026.
- [10] Luciano Ramalho. *Fluent Python: Clear, Concise, and Effective Programming*. O’Reilly Media, 2015.
- [11] Python Software Foundation. Concrete objects layer: List objects, python 3.12 c-api documentation. <https://docs.python.org/3/c-api/list.html>.
- [12] Python Core Developers. listobject.c - cpython source code, github. <https://github.com/python/cpython/blob/main/Objects/listobject.c>.
- [13] Python Software Foundation. sys — system-specific parameters and functions, python 3.12 documentation. <https://docs.python.org/3/library/sys.html>.
- [14] Jochen Liedtke. Improving ipc by kernel design. In *Proceedings of the Fourteenth ACM Symposium on Operating Systems Principles*, pages 175–188, 1993.